



# *Aggregate Functions y Grouping*

Ya que hemos trabajado con funcionalidades esenciales de SQL, es momento de revisar un conjunto de funciones comúnmente conocidas como **funciones de agregación** (*Aggregate Functions*). Estas nos permiten obtener información relevante a partir de un conjunto de datos. También aprenderemos a mostrar información agrupada de acuerdo con valores comunes en ciertos atributos, y a filtrar estos resultados utilizando condiciones específicas.

Antes de comenzar, vamos a ingresar más datos en nuestra tabla de autos para que los ejemplos sean más significativos. Si has estado trabajando con las tablas de los temas anteriores, puedes actualizarlas con los siguientes comandos:

```
INSERT INTO marca VALUES (6, "SUZUKI"), (7, "SUBARU");
```

```
UPDATE submarca SET id_marca = 6 WHERE submarca = "VITARA";
```

```
UPDATE submarca SET id_marca = 7 WHERE submarca = "WRX";
```

```
INSERT INTO submarca VALUES ("FORESTER", 7);
```

```
INSERT INTO submarca VALUES ("SIENNA", 3);
```

```
INSERT INTO submarca VALUES ("FIT", 2);
```

```
INSERT INTO submarca VALUES ("CR-V", 2);
```

```
INSERT INTO auto VALUES
```

```
("3VWHUIHU4535", 2019, "ROJO FLASH" , "MANUAL", 63000),  
("H787N7T34GNG", 2018, "BLANCO" , "MANUAL", 125000),  
("HVNDUD778QER", 2022, "NEGRO" , "AUTOMATICA", 75800),  
("3VWUIHFUHDSF", 2012, "BLANCO" , "AUTOMATICA", 132000),  
("6TDPOKDQ4406", 2023, "BLANCO PERLADO" , "AUTOMATICA", 18500),  
("JDRGERG54564", 2021, "ROJO" , "AUTOMATICA", 213000),  
("9D896GFG9DFG", 2022, "AZUL METALICO" , "AUTOMATICA", 41500),  
("9DTHRUHIEU8", 2024, "AZUL METALICO" , "AUTOMATICA", 37000);
```

La información en nuestras tablas quedará actualizada de la siguiente manera:

```
mysql> SELECT * FROM marca;
+-----+-----+
| id_marca | marca |
+-----+-----+
|         1 | VOLKSWAGEN |
|         2 | HONDA |
|         3 | TOYOTA |
|         4 | FORD |
|         5 | MAZDA |
|         6 | SUZUKI |
|         7 | SUBARU |
+-----+-----+
7 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM submarca;
+-----+-----+
| submarca | id_marca |
+-----+-----+
| POLO | 1 |
| TIGUAN | 1 |
| CIVIC | 2 |
| CR-V | 2 |
| FIT | 2 |
| COROLLA | 3 |
| SIENNA | 3 |
| VITARA | 6 |
| FORESTER | 7 |
| WRX | 7 |
+-----+-----+
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM auto;
```

| vin          | submarca | modelo | color          | transmision | recorrido |
|--------------|----------|--------|----------------|-------------|-----------|
| 3VWU67FR7RE3 | POLO     | 2015   | GRIS PLATINO   | MANUAL      | 125000    |
| HDG6JBSD7FDT | CIVIC    | 2020   | PLATA DIAMANTE | AUTOMATICA  | 42000     |
| 3VWGCISDS7D7 | TIGUAN   | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| 6TDG675E54GY | COROLLA  | 2022   | ROJO           | MANUAL      | 30000     |
| JD76F6SDF7DF | VITARA   | 2016   | BLANCO         | AUTOMATICA  | 213000    |
| 9DTF6S5SG5G5 | WRX      | 2020   | BLANCO         | MANUAL      | 50000     |
| 3VWHUIHU4535 | POLO     | 2019   | ROJO FLASH     | MANUAL      | 63000     |
| H787N7T34GNG | FIT      | 2018   | BLANCO         | MANUAL      | 125000    |
| HVNDUD778QER | CR-V     | 2022   | NEGRO          | AUTOMATICA  | 75800     |
| 3VWUIHFUHSF  | POLO     | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| 6TDPOKDQ4406 | SIENNA   | 2023   | BLANCO PERLADO | AUTOMATICA  | 18500     |
| JDRGERG54564 | VITARA   | 2021   | ROJO           | AUTOMATICA  | 213000    |
| 9D896GFG9DFG | WRX      | 2022   | AZUL METALICO  | AUTOMATICA  | 41500     |
| 9DTHRUIHIEU8 | FORESTER | 2024   | AZUL METALICO  | AUTOMATICA  | 37000     |

4 rows in set (0.00 sec)

Y si recordamos nuestra vista `autos_disponibles`, podemos visualizar toda la información relacionada con nuestros autos:

```
mysql> SELECT * FROM autos_disponibles;
```

| vin          | submarca | marca      | modelo | color          | transmision | recorrido |
|--------------|----------|------------|--------|----------------|-------------|-----------|
| 3VWU67FR7RE3 | POLO     | VOLKSWAGEN | 2015   | GRIS PLATINO   | MANUAL      | 125000    |
| 3VWHUIHU4535 | POLO     | VOLKSWAGEN | 2019   | ROJO FLASH     | MANUAL      | 63000     |
| 3VWUIHFUHSF  | POLO     | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| 3VWGCISDS7D7 | TIGUAN   | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| HDG6JBSD7FDT | CIVIC    | HONDA      | 2020   | PLATA DIAMANTE | AUTOMATICA  | 42000     |
| HVNDUD778QER | CR-V     | HONDA      | 2022   | NEGRO          | AUTOMATICA  | 75800     |
| H787N7T34GNG | FIT      | HONDA      | 2018   | BLANCO         | MANUAL      | 125000    |
| 6TDG675E54GY | COROLLA  | TOYOTA     | 2022   | ROJO           | MANUAL      | 30000     |
| 6TDPOKDQ4406 | SIENNA   | TOYOTA     | 2023   | BLANCO PERLADO | AUTOMATICA  | 18500     |
| JD76F6SDF7DF | VITARA   | SUZUKI     | 2016   | BLANCO         | AUTOMATICA  | 213000    |
| JDRGERG54564 | VITARA   | SUZUKI     | 2021   | ROJO           | AUTOMATICA  | 213000    |
| 9DTHRUIHIEU8 | FORESTER | SUBARU     | 2024   | AZUL METALICO  | AUTOMATICA  | 37000     |
| 9DTF6S5SG5G5 | WRX      | SUBARU     | 2020   | BLANCO         | MANUAL      | 50000     |
| 9D896GFG9DFG | WRX      | SUBARU     | 2022   | AZUL METALICO  | AUTOMATICA  | 41500     |

14 rows in set (0.00 sec)

Ahora que nuestras tablas tienen más datos, podemos continuar.

Si deseas recrear estas tablas en tu instalación para ir siguiendo los ejemplos, en la sección de Archivos adjuntos encontrarás un documento llamado **tablas\_tema4.sql** que contiene las instrucciones para hacerlo. Solo necesitas crear una nueva base de datos y ejecutar todas las instrucciones del archivo.

## Uso de funciones de agregación (SUM, AVG, MAX, MIN, COUNT)

SQL proporciona un conjunto de funciones diseñadas para operar sobre todos los valores de una columna en una tabla. Estas funciones son útiles para **resumir** los valores de múltiples registros. En la siguiente tabla encontrarás algunas de las más comunes:

| Función | Descripción                                                       |
|---------|-------------------------------------------------------------------|
| SUM     | Calcula la suma de los valores en una columna                     |
| AVG     | Calcula el promedio de los valores en una columna o del argumento |
| MAX     | Regresa el valor máximo de los valores en una columna             |
| MIN     | Regresa el valor mínimo de los valores en una columna             |
| COUNT   | Regresa el conteo del conjunto de resultados                      |

A continuación, revisaremos algunos ejemplos de uso de estas funciones, utilizando nuestra base de datos de autos.

## Función SUM

Como ya mencionamos, esta función **calcula la suma de los valores en una columna**. En nuestro caso, tenemos dos columnas con valores numéricos: modelo y recorrido. Aunque no tiene mucho sentido sumar los años de los modelos o los kilometrajes, lo haremos para ilustrar el uso de esta función.

Si tuviéramos el precio de venta de los autos, podría sernos muy útil la función SUM para calcular el valor del inventario, sumando el precio de venta de todos los autos.

La sintaxis del uso de la función SUM es:

```
SELECT SUM(<columna>) FROM <tabla>;
```

Para nuestro ejemplo, utilizamos:

```
SELECT SUM(modelo) FROM autos_disponibles;
```

```
mysql> SELECT SUM(modelo) FROM autos_disponibles;
+-----+
| SUM(modelo) |
+-----+
|      28266 |
+-----+
1 row in set (0.00 sec)
```

```
SELECT SUM(recorrido) FROM autos_disponibles;
```

```
mysql> SELECT SUM(recorrido) FROM autos_disponibles;
+-----+
| SUM(recorrido) |
+-----+
|      1297800 |
+-----+
1 row in set (0.00 sec)
```

También podemos calcular la suma de alguna columna, pero filtrando datos. Por ejemplo, si queremos conocer el recorrido de los autos de la marca TOYOTA, podemos escribir nuestra consulta como:

```
SELECT SUM(recorrido) FROM autos_disponibles WHERE marca= "TOYOTA";
```

Podemos corroborar de manera muy sencilla que la suma del kilometraje de los autos TOYOTA es 48500:

```
mysql> select * from autos_disponibles;
```

| vin          | submarca | marca      | modelo | color          | transmision | recorrido |
|--------------|----------|------------|--------|----------------|-------------|-----------|
| 3VWU67FR7RE3 | POLO     | VOLKSWAGEN | 2015   | GRIS PLATINO   | MANUAL      | 125000    |
| 3VWHUIHU4535 | POLO     | VOLKSWAGEN | 2019   | ROJO FLASH     | MANUAL      | 63000     |
| 3VWUIHFUHSF  | POLO     | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| 3VWGCISDS7D7 | TIGUAN   | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| HDG6JBSD7FDT | CIVIC    | HONDA      | 2020   | PLATA DIAMANTE | AUTOMATICA  | 42000     |
| HVNDUD778QER | CR-V     | HONDA      | 2022   | NEGRO          | AUTOMATICA  | 75800     |
| H787N7T34GNG | FIT      | HONDA      | 2018   | BLANCO         | MANUAL      | 125000    |
| 6TDG675E54GY | COROLLA  | TOYOTA     | 2022   | ROJO           | MANUAL      | 30000     |
| 6TDP0KDQ4406 | SIENNA   | TOYOTA     | 2023   | BLANCO PERLADO | AUTOMATICA  | 18500     |
| JD76F6SDF7DF | VITARA   | SUZUKI     | 2016   | BLANCO         | AUTOMATICA  | 213000    |
| JDRGERG54564 | VITARA   | SUZUKI     | 2021   | ROJO           | AUTOMATICA  | 213000    |
| 9DTHRUHIEU8  | FORESTER | SUBARU     | 2024   | AZUL METALICO  | AUTOMATICA  | 37000     |
| 9DTF6S5SG5G5 | WRX      | SUBARU     | 2020   | BLANCO         | MANUAL      | 50000     |
| 9D896GFG9DFG | WRX      | SUBARU     | 2022   | AZUL METALICO  | AUTOMATICA  | 41500     |

```
14 rows in set (0.00 sec)
```

```
mysql> SELECT SUM(recorrido) FROM autos_disponibles WHERE marca= "TOYOTA";
```

| SUM(recorrido) |
|----------------|
| 48500          |

```
1 row in set (0.00 sec)
```

## Función AVG

Esta función **calcula el promedio de los valores en una columna**. En el caso de nuestra base de datos, podemos obtener el promedio del año modelo o del kilometraje de los autos.

La sintaxis del uso de la función AVG es:

```
SELECT AVG(<columna>) FROM <tabla>;
```

Para ver cuál es el promedio del modelo de los autos que trabajamos, la consulta quedaría:

```
SELECT AVG(modelo) FROM autos_disponibles;
```

```
mysql> SELECT AVG(modelo) FROM autos_disponibles;
+-----+
| AVG(modelo) |
+-----+
|    2019.0000 |
+-----+
1 row in set (0.00 sec)
```

Para ver cuál es el promedio del kilometraje de los autos que trabajamos, la consulta es:

```
SELECT AVG(recorrido) FROM autos_disponibles;
```

```
mysql> SELECT AVG(recorrido) FROM autos_disponibles;
+-----+
| AVG(recorrido) |
+-----+
|    92700.0000 |
+-----+
1 row in set (0.00 sec)
```



Para ver cuál es el promedio del modelo de los autos TOYOTA que trabajamos, la consulta quedaría:

```
SELECT AVG(modelo) FROM autos_disponibles WHERE marca= "TOYOTA";
```

```
mysql> SELECT AVG(modelo) FROM autos_disponibles WHERE marca= "TOYOTA";
+-----+
| AVG(modelo) |
+-----+
| 2022.5000 |
+-----+
1 row in set (0.00 sec)
```

Para ver cuál es el promedio del recorrido de los autos TOYOTA que trabajamos, la consulta sería:

```
SELECT AVG(recorrido) FROM autos_disponibles WHERE marca= "TOYOTA";
```

```
mysql> SELECT AVG(recorrido) FROM autos_disponibles WHERE marca= "TOYOTA";
+-----+
| AVG(recorrido) |
+-----+
| 24250.0000 |
+-----+
1 row in set (0.00 sec)
```

## Funciones MAX y MIN

Estas funciones **permiten obtener el valor más alto o más bajo de una columna**, respectivamente. Por ejemplo, si queremos consultar cuál es el modelo más reciente de auto que trabajamos, esto significa consultar el máximo de la columna modelo:

```
SELECT MAX(modelo) FROM autos_disponibles;
```

```
mysql> SELECT MAX(modelo) FROM autos_disponibles;
+-----+
| MAX(modelo) |
+-----+
|         2024 |
+-----+
1 row in set (0.00 sec)
```

Para saber cuál es el auto con el menor kilometraje con el que contamos, es necesario consultar el mínimo de la columna recorrido:

```
SELECT MIN(recorrido) FROM autos_disponibles;
```

```
mysql> SELECT MIN(recorrido) FROM autos_disponibles;
+-----+
| MIN(recorrido) |
+-----+
|          18500 |
+-----+
1 row in set (0.00 sec)
```

Y si queremos consultar cuál es el auto de marca HONDA o TOYOTA con menor kilometraje, utilizamos:

```
SELECT MIN(recorrido) FROM autos_disponibles WHERE marca = "HONDA" OR marca = "TOYOTA";
```

```
mysql> SELECT MIN(recorrido) FROM autos_disponibles WHERE marca = "HONDA" OR marca = "TOYOTA";
+-----+
| MIN(recorrido) |
+-----+
|          18500 |
+-----+
1 row in set (0.01 sec)
```

Ahora, para ver cuál es el auto VOLKSWAGEN más antiguo que tenemos disponible, usamos:

```
SELECT MIN(modelo) FROM autos_disponibles WHERE marca = "VOLKSWAGEN";
```

```
mysql> SELECT MIN(modelo) FROM autos_disponibles WHERE marca = "VOLKSWAGEN";
+-----+
| MIN(modelo) |
+-----+
|          2012 |
+-----+
1 row in set (0.00 sec)
```

## Función COUNT

En algunas situaciones, es posible que solo nos interese conocer el número de registros que cumplen una cierta condición, no cuáles. Aquí es útil la función COUNT, que **devuelve la cantidad de registros en una consulta**. La sintaxis es:

```
SELECT COUNT (*) FROM <tabla>
```

En nuestro ejemplo de la agencia, si queremos saber cuántos autos tenemos, podemos consultarlo de este modo:

```
SELECT COUNT(*) FROM autos_disponibles;
```

```
mysql> SELECT COUNT(*) FROM autos_disponibles;
+-----+
| COUNT(*) |
+-----+
|      14 |
+-----+
1 row in set (0.00 sec)
```

Si queremos saber cuántos autos HONDA, TOYOTA o SUBARU tenemos, podemos ejecutar:

```
SELECT COUNT(*) FROM autos_disponibles
```

```
WHERE marca = "HONDA" OR marca = "TOYOTA" OR marca = "SUBARU";
```

```
mysql> SELECT COUNT(*) FROM autos_disponibles
-> WHERE marca = "HONDA" OR marca = "TOYOTA" OR marca = "SUBARU";
+-----+
| COUNT(*) |
+-----+
|        8 |
+-----+
1 row in set (0.00 sec)
```

Si queremos saber cuántos autos tienen transmisión automática, usamos:

```
SELECT COUNT(*) FROM autos_disponibles  
WHERE transmission = "AUTOMATICA" ;
```

```
mysql> SELECT COUNT(*) FROM autos_disponibles  
-> WHERE transmission = "AUTOMATICA" ;  
+-----+  
| COUNT(*) |  
+-----+  
|          9 |  
+-----+  
1 row in set (0.00 sec)
```

Para ver cuántos autos son de color blanco, podemos utilizar:

```
SELECT COUNT(*) FROM autos_disponibles  
WHERE color LIKE "BLANCO%" ;
```

```
mysql> SELECT COUNT(*) FROM autos_disponibles  
-> WHERE color LIKE "BLANCO%" ;  
+-----+  
| COUNT(*) |  
+-----+  
|          6 |  
+-----+  
1 row in set (0.01 sec)
```

**Nota:** COUNT (\*) cuenta **todas** las filas, incluso si contienen valores nulos, mientras que COUNT(nombre\_columna) ignora los valores nulos en esa columna.

Estas funciones no son las únicas disponibles. Si deseas explorar más funciones agregadas, puedes consultar la sección de Material adicional, donde encontrarás enlaces para acceder a la documentación oficial de MySQL.

## Agrupación de datos con `GROUP BY` y filtrado de grupos con `HAVING`

En el subtema anterior, vimos cómo realizar una consulta que mostrara la suma del kilometraje de los autos TOYOTA o el número de autos de una marca específica. Seguramente te preguntaste si era posible hacer este tipo de consultas, pero para cada una de las marcas, por ejemplo: ver la suma del kilometraje de los autos de todas las marcas, o conocer el número de autos que tenemos de cada marca. Este tipo de consultas se pueden hacer utilizando el comando `GROUP BY`.

El comando `GROUP BY` en SQL se utiliza junto con la instrucción `SELECT` para resumir información en grupos. Crea filas que contienen datos resumidos de varias filas que comparten un mismo valor en algún atributo. Esta instrucción se coloca después de un `SELECT` que puede llevar una condición `WHERE`. De manera opcional, también se pueden incluir las instrucciones `HAVING` y `ORDER BY`.

La sintaxis de la instrucción `GROUP BY` es:

```
SELECT <atributos>
FROM <tabla>
WHERE <condición>
GROUP BY <columna(s)>
ORDER BY <columna(s)>;
```

La instrucción `GROUP BY` se usa con frecuencia junto con las funciones de agregación que ya comentamos previamente. Veamos su uso con algunos ejemplos a continuación.

Primero, escribamos una consulta que muestre la suma del kilometraje de los autos de cada marca. Lo que necesitamos aquí es agrupar la información por marca y sumar la columna `recorrido` para cada uno de esos grupos. La consulta quedaría así:

```
SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca;
```

```
mysql> SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca;
+-----+-----+
| marca      | SUM(recorrido) |
+-----+-----+
| VOLKSWAGEN |         452000 |
| HONDA      |         242800 |
| TOYOTA     |          48500 |
| SUZUKI     |         426000 |
| SUBARU     |         128500 |
+-----+-----+
5 rows in set (0.01 sec)
```

De manera opcional, podemos pedirle a la consulta que ordene los resultados por marca:

```
SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca ORDER BY marca;
```

```
mysql> SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca ORDER BY marca;
+-----+-----+
| marca      | SUM(recorrido) |
+-----+-----+
| HONDA      |         242800 |
| SUBARU     |         128500 |
| SUZUKI     |         426000 |
| TOYOTA     |          48500 |
| VOLKSWAGEN |         452000 |
+-----+-----+
5 rows in set (0.05 sec)
```

O por la suma del recorrido:

```
SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca ORDER BY SUM(recorrido);
```

```
mysql> SELECT marca, SUM(recorrido) FROM autos_disponibles GROUP BY marca ORDER BY SUM(recorrido);
+-----+-----+
| marca      | SUM(recorrido) |
+-----+-----+
| TOYOTA      | 48500           |
| SUBARU      | 128500          |
| HONDA       | 242800          |
| SUZUKI      | 426000          |
| VOLKSWAGEN  | 452000          |
+-----+-----+
5 rows in set (0.00 sec)
```

Otro ejemplo puede ser consultar el número de autos disponibles por marca:

```
SELECT marca, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY marca
ORDER BY marca;
```

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> ORDER BY marca;
+-----+-----+
| marca      | unidades disponibles |
+-----+-----+
| HONDA      | 3                     |
| SUBARU     | 3                     |
| SUZUKI     | 2                     |
| TOYOTA     | 2                     |
| VOLKSWAGEN | 4                     |
+-----+-----+
5 rows in set (0.00 sec)
```



```
SELECT marca, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY marca
ORDER BY COUNT(*);
```

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> ORDER BY COUNT(*);
```

| marca      | unidades disponibles |
|------------|----------------------|
| TOYOTA     | 2                    |
| SUZUKI     | 2                    |
| HONDA      | 3                    |
| SUBARU     | 3                    |
| VOLKSWAGEN | 4                    |

5 rows in set (0.01 sec)

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> ORDER BY COUNT(*) DESC;
```

| marca      | unidades disponibles |
|------------|----------------------|
| VOLKSWAGEN | 4                    |
| HONDA      | 3                    |
| SUBARU     | 3                    |
| TOYOTA     | 2                    |
| SUZUKI     | 2                    |

5 rows in set (0.00 sec)

También podemos ver el promedio del modelo de los autos que se tienen por marca:

```
SELECT marca, AVG(recorrido) AS "promedio kilometraje"  
FROM autos_disponibles  
GROUP BY marca  
ORDER BY marca;
```

```
mysql> SELECT marca, AVG(recorrido) AS "promedio kilometraje"  
-> FROM autos_disponibles  
-> GROUP BY marca  
-> ORDER BY marca;  
  
+-----+-----+  
| marca          | promedio kilometraje |  
+-----+-----+  
| HONDA          | 80933.3333           |  
| SUBARU         | 42833.3333           |  
| SUZUKI         | 213000.0000          |  
| TOYOTA         | 24250.0000           |  
| VOLKSWAGEN     | 113000.0000          |  
+-----+-----+  
5 rows in set (0.00 sec)
```

Otro ejemplo es ver cuántos autos se tienen de cada modelo, y ordenarlos por cantidad:

```
SELECT modelo, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY modelo
ORDER BY COUNT(*);
```

```
mysql> SELECT modelo, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY modelo
-> ORDER BY COUNT(*);
```

| modelo | unidades disponibles |
|--------|----------------------|
| 2015   | 1                    |
| 2019   | 1                    |
| 2018   | 1                    |
| 2023   | 1                    |
| 2016   | 1                    |
| 2021   | 1                    |
| 2024   | 1                    |
| 2012   | 2                    |
| 2020   | 2                    |
| 2022   | 3                    |

```
10 rows in set (0.01 sec)
```

0 por modelo:

```
SELECT modelo, COUNT(*) AS "unidades disponibles"  
  FROM autos_disponibles  
 GROUP BY modelo  
 ORDER BY modelo;
```

```
mysql> SELECT modelo, COUNT(*) AS "unidades disponibles"  
->      FROM autos_disponibles  
->      GROUP BY modelo  
->      ORDER BY modelo;
```

| modelo | unidades disponibles |
|--------|----------------------|
| 2012   | 2                    |
| 2015   | 1                    |
| 2016   | 1                    |
| 2018   | 1                    |
| 2019   | 1                    |
| 2020   | 2                    |
| 2021   | 1                    |
| 2022   | 3                    |
| 2023   | 1                    |
| 2024   | 1                    |

```
10 rows in set (0.00 sec)
```

Retomando la consulta de las unidades disponibles por marca:

```
SELECT marca, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY marca
ORDER BY COUNT(*);
```

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> ORDER BY COUNT(*);
```

| marca      | unidades disponibles |
|------------|----------------------|
| TOYOTA     | 2                    |
| SUZUKI     | 2                    |
| HONDA      | 3                    |
| SUBARU     | 3                    |
| VOLKSWAGEN | 4                    |

```
5 rows in set (0.00 sec)
```

Supongamos ahora que queremos saber de qué marcas tenemos 2 o más unidades. Lo más intuitivo sería pensar que podemos usar `WHERE`:

```
SELECT marca, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY marca
WHERE COUNT(*) > 2
ORDER BY COUNT(*);
```

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> WHERE COUNT(*) > 2
-> ORDER BY COUNT(*);
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE COUNT(\*) > 2'

Sin embargo, esto **no es posible**, ya que la información ya está agrupada. En su lugar, debemos usar la instrucción `HAVING` **para especificar condiciones que deben cumplir los grupos**.

**Recuerda:** `WHERE` se utiliza para filtrar **filas individuales** en una tabla o en una vista, mientras que `HAVING` se usa para filtrar **conjuntos** de datos ya agrupados.

La consulta correcta sería:

```
SELECT marca, COUNT(*) AS "unidades disponibles"
FROM autos_disponibles
GROUP BY marca
HAVING COUNT(*) > 2
ORDER BY COUNT(*) ;
```

```
mysql> SELECT marca, COUNT(*) AS "unidades disponibles"
-> FROM autos_disponibles
-> GROUP BY marca
-> HAVING COUNT(*) > 2
-> ORDER BY COUNT(*) ;

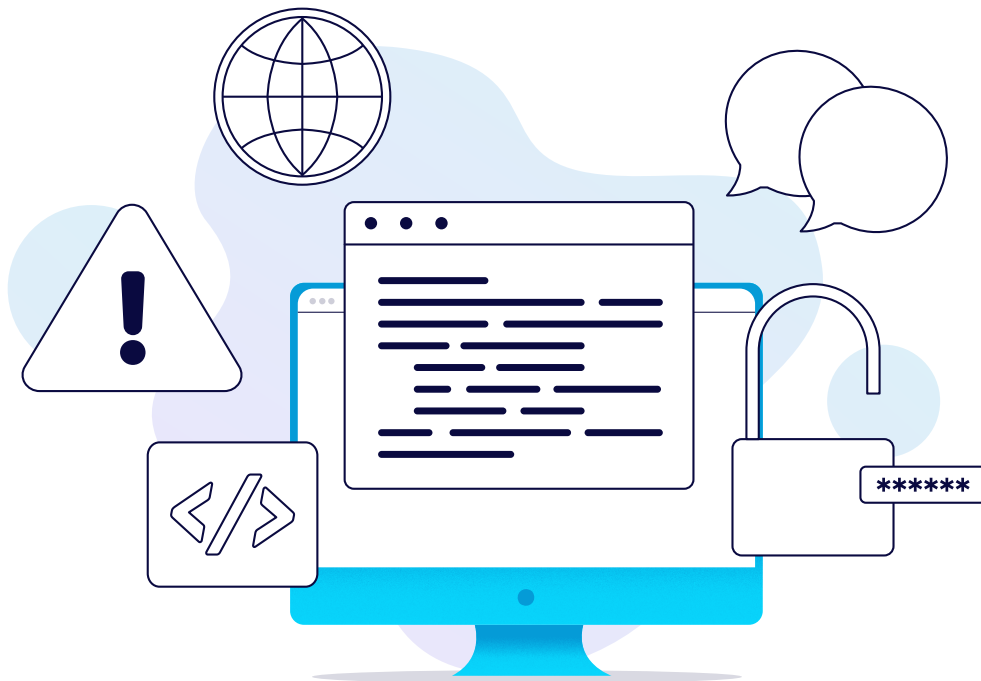
+-----+-----+
| marca      | unidades disponibles |
+-----+-----+
| HONDA      | 3                    |
| SUBARU     | 3                    |
| VOLKSWAGEN | 4                    |
+-----+-----+
3 rows in set (0.00 sec)
```

Al iniciar este subtema, mencionamos que la sintaxis para usar GROUP BY es:

```
SELECT <atributos>  
FROM <tabla>  
WHERE <condición>  
GROUP BY <columna(s)>  
ORDER BY <columna(s)>;
```

Y si queremos usar HAVING, el orden correcto es colocarlo **después de** GROUP BY **y antes de** ORDER BY, de este modo:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```



Para concluir, veamos un último ejemplo. Imagina que queremos mostrar todas las marcas de vehículos cuyo modelo más antiguo sea 2020 o más reciente:

```
SELECT marca, MIN(modelo) AS "minimo modelo"  
FROM autos_disponibles  
GROUP BY marca  
HAVING MIN(modelo) >= 2020  
ORDER BY MIN(modelo);
```

```
mysql> SELECT marca, MIN(modelo) AS "minimo modelo"  
-> FROM autos_disponibles  
-> GROUP BY marca  
-> HAVING MIN(modelo) >= 2020  
-> ORDER BY MIN(modelo);  
+-----+-----+  
| marca | minimo modelo |  
+-----+-----+  
| SUBARU | 2020 |  
| TOYOTA | 2022 |  
+-----+-----+  
2 rows in set (0.07 sec)
```



Podemos confirmar esta información con la vista de `autos_disponibles`:

```
mysql> SELECT * FROM autos_disponibles;
```

| vin          | submarca | marca      | modelo | color          | transmision | recorrido |
|--------------|----------|------------|--------|----------------|-------------|-----------|
| 3VWU67FR7RE3 | POLO     | VOLKSWAGEN | 2015   | GRIS PLATINO   | MANUAL      | 125000    |
| 3VWHUIHU4535 | POLO     | VOLKSWAGEN | 2019   | ROJO FLASH     | MANUAL      | 63000     |
| 3VWUIHFUHSF  | POLO     | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| 3VWGCISDS7D7 | TIGUAN   | VOLKSWAGEN | 2012   | BLANCO         | AUTOMATICA  | 132000    |
| HDG6JBSD7FDT | CIVIC    | HONDA      | 2020   | PLATA DIAMANTE | AUTOMATICA  | 42000     |
| HVNDUD778QER | CR-V     | HONDA      | 2022   | NEGRO          | AUTOMATICA  | 75800     |
| H787N7T34GNG | FIT      | HONDA      | 2018   | BLANCO         | MANUAL      | 125000    |
| 6TDG675E54GY | COROLLA  | TOYOTA     | 2022   | ROJO           | MANUAL      | 30000     |
| 6TDP0KDQ4406 | SIENNA   | TOYOTA     | 2023   | BLANCO PERLADO | AUTOMATICA  | 18500     |
| JD76F6SDF7DF | VITARA   | SUZUKI     | 2016   | BLANCO         | AUTOMATICA  | 213000    |
| JDRGERG54564 | VITARA   | SUZUKI     | 2021   | ROJO           | AUTOMATICA  | 213000    |
| 9DTHRUHIEU8  | FORESTER | SUBARU     | 2024   | AZUL METALICO  | AUTOMATICA  | 37000     |
| 9DTF6S5SG5G5 | WRX      | SUBARU     | 2020   | BLANCO         | MANUAL      | 50000     |
| 9D896GFG9DFG | WRX      | SUBARU     | 2022   | AZUL METALICO  | AUTOMATICA  | 41500     |

14 rows in set (0.00 sec)

Recuerda que, si deseas revisar esto más a detalle, en la sección de Material adicional encontrarás enlaces para acceder a la documentación oficial de MySQL.

## ¶ Para terminar...

A lo largo de este tema, aprendimos que **SQL no solo sirve para almacenar y consultar datos**, sino que también nos permite **resumir, comparar y encontrar valores importantes** mediante funciones de agregación como `SUM`, `COUNT`, `AVG`, `MAX`, `MIN`, junto con el uso de `GROUP BY` y `HAVING`.

Más allá de la sintaxis y los ejemplos revisados, este tipo de análisis tiene un valor importante: nos permite convertir los datos en información útil para la toma de decisiones.

Antes de continuar, reflexiona:

- ¿Qué decisiones has tomado en tu vida o trabajo basándote solo en la intuición, sin revisar datos concretos?
- ¿Cómo podrías aplicar estas funciones de SQL para analizar tu entorno? Por ejemplo: tus gastos mensuales, tus hábitos de estudio o el rendimiento de un equipo o negocio.
- ¿Se te ocurren escenarios donde podrías agrupar información y analizar promedios, totales o máximos en tu área de interés o en tu día a día?

La capacidad de **agrupar, comparar y analizar información** no es exclusiva del mundo de las bases de datos. Es una habilidad esencial en el mundo actual.

Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.

## Material adicional

### Documentación oficial de MySQL:

MySQL. (2024). *Aggregate Functions* [Funciones de agregación]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions-and-modifiers.html>

MySQL. (2024). *Aggregate Function Descriptions* [Descripciones de funciones de agregación]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html>

MySQL. (2024). *MySQL Handling of GROUP BY* [Manejo de GROUP BY en MySQL]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/group-by-handling.html>

### Documentación de Tutorials Point:

Tutorials Point. (s.f.). *SQL - Aggregate Functions* [SQL - Funciones de agregación]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-aggregate-functions.htm>

Tutorials Point. (s.f.). *SQL - Group By Clause* [SQL – Cláusula Group By]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-group-by.htm>

Tutorials Point. (s.f.). *SQL - Having Clause* [SQL – Cláusula Having]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-having-clause.htm>

**Nota:** Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 23 de abril de 2025 y su disponibilidad depende del autor.

## | Glosario

- **Agrupación de datos (*Grouping*):** Es el proceso mediante el cual se organizan filas de una tabla que comparten un valor común en una columna específica, permitiendo así realizar operaciones de resumen sobre cada grupo.
- **Consulta agregada:** Es una instrucción SQL que utiliza funciones como `SUM`, `COUNT` o `AVG` para calcular valores sobre conjuntos de datos, en lugar de procesar fila por fila.
- **Filtrado de grupos (*HAVING*):** Es una cláusula que permite aplicar condiciones a los grupos ya formados por `GROUP BY`, filtrando solo aquellos que cumplen con ciertos criterios. No debe confundirse con `WHERE`, que filtra filas antes de agrupar.
- **Funciones de resumen:** Son operaciones que permiten obtener estadísticas sobre los datos, como totales, promedios o valores mínimos y máximos, facilitando el análisis y toma de decisiones.
- **Sintaxis de SQL:** Se refiere a las reglas que definen cómo deben estructurarse las instrucciones en SQL para que la base de datos las entienda y procese correctamente.

## Bibliografía

Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of database systems* [Fundamentos de los sistemas de bases de datos] (7th ed.). Pearson Education.

MySQL. (2024). *Aggregate Functions* [Funciones de agregación]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions-and-modifiers.html>

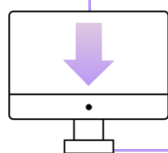
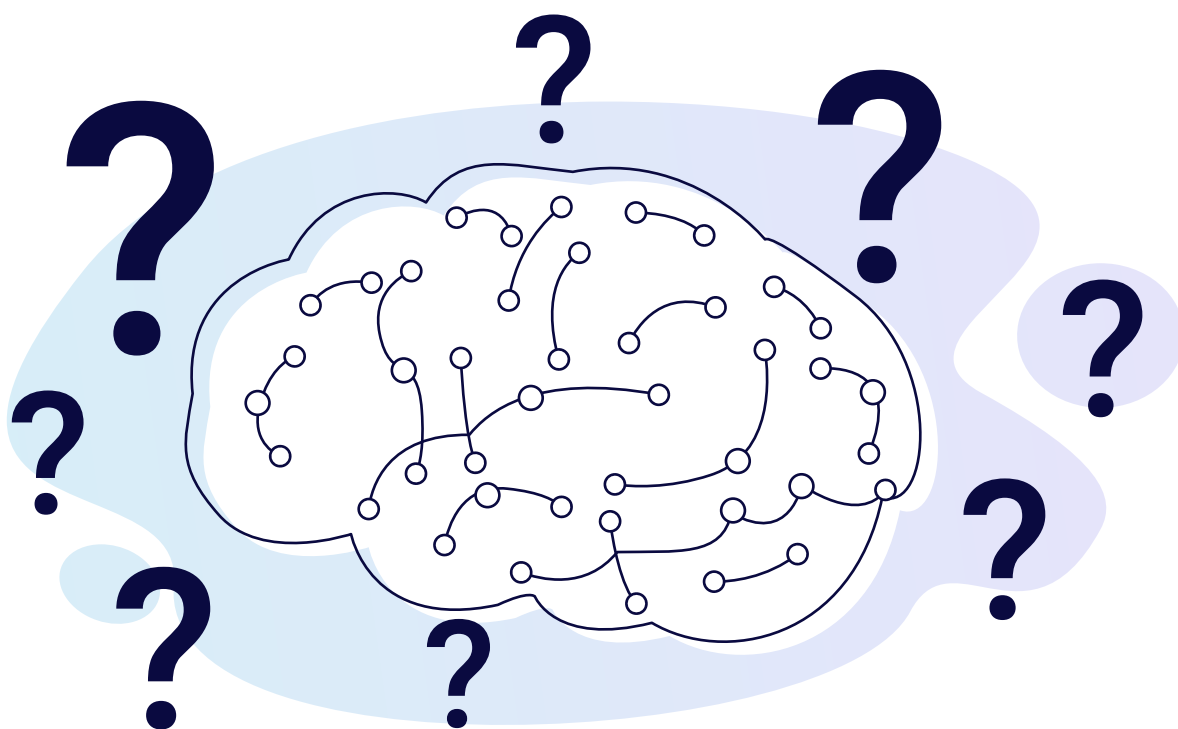
MySQL. (2024). *Aggregate Function Descriptions* [Descripciones de funciones de agregación]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/aggregate-functions.html>

MySQL. (2024). *MySQL Handling of GROUP BY* [Manejo de GROUP BY en MySQL]. MySQL 8.4 Reference Manual. Oracle. Recuperado el 23 de abril de 2025 de: <https://dev.mysql.com/doc/refman/8.4/en/group-by-handling.html>

Tutorials Point. (s.f.). *SQL - Aggregate Functions* [SQL - Funciones de agregación]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-aggregate-functions.htm>

Tutorials Point. (s.f.). *SQL - Group By Clause* [SQL – Cláusula Group By]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-group-by.htm>

Tutorials Point. (s.f.). *SQL - Having Clause* [SQL – Cláusula Having]. SQL Function Reference. Tutorials Point. Recuperado el 23 de abril de 2025 de: <https://www.tutorialspoint.com/sql/sql-having-clause.htm>



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.

**Nota:** Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 23 de abril de 2025 y su disponibilidad depende del autor.